Preprocessing Pipeline for Monthly Return Prediction

1 Notation

Let $i \in \{1, \ldots, N_c\}$ index companies and let t index months. Each row in the raw file corresponds to a company–month pair (i, t). We denote the target by $y_{i,t}$ (the column `monthly_gross_return`), a gross return where values > 1 indicate gains and values < 1 indicate losses. For interpretation one may define the net return $r_{i,t} = y_{i,t} - 1$, but the pipeline operates on $y_{i,t}$. The raw feature vector for a row is $\mathbf{x}_{i,t} \in \mathbb{R}^d$. Define the design matrix $X \in \mathbb{R}^{N \times d}$ where row index $r = r(i, t)$ maps to a specific pair (i, t), so $X_{r,:} = \mathbf{x}_{i,t}$. Missingness indicators are binary features $m_{i,t,j} = \mathbb{1}[x_{i,t,j} \text{ is missing}]$.

2 Raw Data Schema

The raw data are a panel of company–month observations. Key fields are:

- **Identifiers:** `co_code` (company id) and `Month` (YYYY-MM timestamp).
- **Target:** `monthly_gross_return`.
- **Market variables:** `mktcap`, `price`, `lag_mv`, `BM_sep`.
- **Accounting variables:** `equity_bv_on_stkdate`, `fy_book_value`, `sa_net_worth`, `sa_total_assets`, `sa_sales`, `sa_cost_of_goods_sold`, `sa_total_interest_exp`, `sa_selling_distribution_exp`, `lagged_book_equity`, `lagged_assets`, `lagged2_assets`.
- **Factor-style signals:** `OpProf`, `Inv`, `Momentum`.
- **Categorical labels:** `Size_Label` (S/B), `BM_Label` (G/N/V), `OpProf_Label` (W/N/R), `Inv_Label` (A/N/C), `Mom_Label` (L/N/W).
- **Calendar support:** `Year`, `Corrected_Year`, `Corrected_Month`, `Month_annual`.

Any remaining columns are treated as numeric features.

3 Pipeline Overview

The preprocessing mapping is

$$\texttt{factor\_data.csv} \xrightarrow{\text{type coercion + cleaning}} \xrightarrow{\text{lag/rolling features}} \xrightarrow{\text{missing indicators}} \xrightarrow{\text{split assignment}} \{\texttt{train}, \texttt{validation}, \texttt{test}\}.$$

4 Type Coercion and Cleaning

- **Column normalization:** column names are stripped of whitespace.
- **Date parsing:** `Month` is parsed to a datetime. Rows with invalid `Month` are dropped.
- **Identifiers:** `co_code` is coerced to nullable integer.
- **Target:** `monthly_gross_return` is coerced to float.
- **Categorical labels:** categorical columns are coerced to strings, stripped, and empty tokens (`empty string`, `nan`, `<NA>`) are converted to missing values.
- **Numeric features:** all remaining non-categorical, non-identifier columns are coerced to floats; non-numeric values become missing.

Raw return columns `lag_ret` and `log_ret` are removed to avoid leakage. All engineered return history is derived from the target $y_{i,t}$.

5 Leakage-Safe Feature Engineering

All time features are computed within each company after sorting by (`co_code`, `Month`).

5.1 Lag Features

For lag steps $k \in \{1, 3, 6\}$,

$$\text{ret\_lag}_k(i,t) = y_{i,t-k}.$$

These lags use only past values and therefore respect causal ordering.

5.2 Rolling Statistics

Let $w \in \{3, 6\}$ denote the rolling window length and define the available past returns as $\{y_{i,t-1}, y_{i,t-2}, \dots\}$. For each window, the pipeline computes

$$\mu_{i,t}^{(w)} = \frac{1}{m} \sum_{s=1}^{m} y_{i,t-s}, \quad m = \min\{w,\ \#\text{available past returns}\},$$

and

$$\sigma_{i,t}^{(w)} = \sqrt{\frac{1}{m-1} \sum_{s=1}^{m} (y_{i,t-s} - \mu_{i,t}^{(w)})^2}, \quad m \geq 2.$$

The script also records the count of available values in the 3-month window,

$$\text{ret\_rolling\_count\_3m}(i,t) = \#\{s \in \{1,2,3\} : y_{i,t-s} \text{ observed}\}.$$

5.3 Month Index

The monotone calendar index is

$$\text{month\_index}_{i,t} = 12 \cdot \text{year}(t) + \text{month}(t).$$

6 Missingness Indicators

For every column except `co_code` and `Month`, a missingness indicator is added:

$$m_{i,t,j} = \mathbb{1}[x_{i,t,j} \text{ is missing}].$$

This keeps the original missing values while making their presence explicit to downstream models.

7 Split Assignment

Two split modes are supported.

7.1 Time-Based Split

Let t^* be the split date for each row. If `split_date_source=corrected` and corrected fields exist, then t^* is formed from `Corrected_Year` and `Corrected_Month`; otherwise $t^* =$ `Month`. Given cutoffs τ_{train} and τ_{val}, define

$$\text{split}(i,t) = \begin{cases} \text{train}, & t^* \le \tau_{\text{train}}, \\ \text{validation}, & \tau_{\text{train}} < t^* \le \tau_{\text{val}}, \\ \text{test}, & t^* > \tau_{\text{val}}. \end{cases}$$

7.2 Company-Based Split

Companies are assigned to splits in aggregate. Let H_i be the number of distinct months for company i. The validation set is drawn first from the eligible set $\{i : H_i \ge H_{\min}\}$; if insufficient, the remaining slots are filled with shorter-history companies. Remaining companies are split into train and test to match the requested shares. A fixed random seed ensures reproducibility and each company appears in exactly one split.

8 Leakage-Safe Test Outputs

For the test split, the target and raw return columns are masked to avoid inference leakage:

$$y_{i,t} \leftarrow \text{NaN} \quad \text{for test rows.}$$

The original values are written to `test_targets.csv` for evaluation. Any missing indicators for masked columns are updated to reflect the new missing status.

9 Holdout Company (`co_code = 194`)

In addition to the train/validation/test splits, company `194` is removed from the initial dataset and treated as a standalone holdout. This produces a clean, out-of-sample company-level evaluation set that never appears in model training or split assignment. The holdout rows follow the same preprocessing steps (type coercion, leakage-safe feature engineering, and missingness indicators) before inference and reporting.

10 Outputs

The script writes:

- `processed_full.csv`: full dataset with engineered features and `split_group` labels.
- `train.csv`, `validation.csv`, `test.csv`: split subsets, with the test target masked.
- `test_targets.csv`: true target (and any raw return columns if present) for the test rows.
- `metadata.json`: row counts, split counts, categorical columns, and a feature column list (excluding identifiers, dates, target, and missing-indicator columns).

The resulting tables are ready for training and inference: CatBoost consumes the tabular features directly, while the CNN stage builds short temporal sequences from the same leakage-safe history.